WFMS: Web File Management as A Service

Quansong Deng,Yong Zhang, Chunxiao Xing
Web and Software Technology (WeST) R&D center
Tsinghua University
Beijing, China
bonndeng@163.com

***Abstract*—generally, every website provides its own resource upload and download modules for users to share resources. However, the resources uploaded to websites' servers are often lack of unified and professional management. What's more, when the amount of user using the upload and download modules increases, it will put a big pressure on the website's server in term of bandwidth. To solve these problems, this paper designed and implemented a Web resource supporting platform—WFMS (Web File Management System) which can provide a unified interface for multiple websites to upload, download and manage resources professionally, and decrease the developing costs and bandwidth pressure as well.**

Keywords-WFMS(Web File Management System); unified and professional management; developing cost; badwidth pressure

I. INTRODUCTION

As the development of computer technology and network, people use internet more than ever to communicate with each other. With the huge demand of sharing in Web communities, more and more users and applications require to upload, download and manage their own sharing resources. As far as China is concerned, in the 21st Statistical Report of the Status of China Internet Network Development, China Internet Network Information Center (CNNIC) showed, the number of Chinese netizen has reached 210 million by Nov, 2007[1]. As the amount of user increases so fast, sharing resources that Web applications pile up on the Web are increasingly enormous.

Generally, every website provides its own resource upload and download modules for users to share resources. But the resources uploaded to the websites' servers are often lack of unified and professional management. What's more, when the amount of user using the upload and download modules increases, it will put a big pressure on the website's server in term of bandwidth.

To solve these problems, this paper designed and implemented a Web resource supporting platform—(WFMS, Web File Management System) which can provide a unified interface for multiple websites to upload, download and manage resources professionally, and decrease the developing costs and bandwidth pressure. After investigating many solutions for file organization, this paper implemented the static search structure organization method in the system, compared the experimental results from structures of different numbers of layers and gave some performance analysis, which lays the foundation of the final application of our system.

The paper is organized as follows: In section II, we introduce some algorithms about Web file organization method which can be divided into two kinds briefly—dynamic and static search structure organization method. In section III and IV, we will present our design and implementation of the system architecture and functional modules. Then we will design and work on some experiments in section V and after that we will come to our conclusions and describe the future work in section VI.

II. WEB FILE ORGANIZATION ALGORITHMS

WFMS is an application system based on Web, and it has to be quick when users use it to upload, download resources and manage them effectively. To reach the goal, we have to organize all the files uploaded to the server in the first place so that it can be accessed fast later.

A. UUID (Universally Unique Identifier)

A UUID is 128 bits long, and can guarantee uniqueness across space and time. UUID were originally used in the Apollo Network Computing System and later in the Open Software Foundation's (OSF) Distributed Computing Environment (DCE), and then in Microsoft Windows platforms.

In WFMS, in order to be easy to manage, we assign each uploaded file a unique identifier, and that is UUID. Our UUID is generated by Java, after preprocess, each UUID is a 32 bits long hexadecimal string, here's an example: 0f36aa4753114493a09453e4b0d5fa37.

Next, we will briefly introduce two mainly kinds of Web file organization method.

B. Dynamic Search Structure Organization Method

1) Dynamic Search Structure Organization Method

To keep high efficiency of searching, the search structure is automatically adjusted after performing operations like insertion or deletion, and the structure itself may change. This kind of organization method is called dynamic search structure organization method [2].

2) Definition of AVL Tree

AVL tree [3] is:

- A empty tree, or
- A binary search tree which satisfies: its left and right sub trees are also AVL tree, and the height difference of the left and right sub trees is no more than 1.

3) File Organization Method Based On AVL Tree

978-0-7695-3494-7/08 $25.00 © 2008 IEEE
DOI 10.1109/ISISE.2008.124

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on March 05,2024 at 01:34:46 UTC from IEEE Xplore. Restrictions apply.

a) Algorithm Description: We use AVL Tree as our file indexing structure, and the UUID uniquely corresponding to the uploaded file is used as the key for indexing.

b) Discussion: Assume the layer of the root node is layer 1, and the height is 0, so we have:

- *Node number:* In layer i, we have 2^i nodes (if the tree is full).
- *The restriction of the layer number:* Given the requirement of high capacity, if we have 21 layers altogether and the tree is full, then the total number of nodes is $2^0+2^1+2^2+\ldots+2^{20}=2^{21}-1=2097151$, which is already quite a huge amount.
- *Time Complexity:* It is up to the property of AVL tree. If there are n nodes in the tree, the time complexity of inserting, deleting or searching a specified node is O(logn). Considering the maximum layer number restriction which is mentioned above, we need at most 21 comparisons to do such operations.

4) Problems and Possible Solutions

a) To maintain the structure of AVL tree so as to get high efficiency of searching, each insert or delete operation may cause the tree to change the tree structure by rotations, and the time complexity is O(logn) as is mentioned above, which is quite unacceptable.

b) Non-Real-Time Insertion and Deletion: When we insert or delete files from our system very frequently, the structure of AVL tree will change very frequently too, which increases the costs of our system to a great extent. To improve the efficiency, we can consider Non-Real-Time insertion and deletion. That is, we set a temporary folder to hold all the newly inserted files, and record the operation information (file information, operation type—insert or delete, and so on), and then we will do the insertion or deletion in batch regularly. Here, "regularly" needs deeply thought and practices. In general, when the data size is not too huge, we can insert or delete in batch every day, and the timing must be some time when our system is relatively idle.

c) Problems of Non-Real-Time Insertion and Deletion:

- Reference (download) the file which is still in the temporary folder and haven't inserted into our AVL tree.
- Reference (download) the file which is already deleted logically and still in the temporary folder.
- Solutions: Create a table in database to record all the insertion and deletion operations, every time we need to reference(download) one file, we have to search it in the operation record table at first, if we don't find it, then we'll search it in the file tree structure instead.

C. Static Search Structure Organization Method

1) Static Search Structure Organization Method

The search structure doesn't change before and after performing operations like insertion and deletion. This kind of organization method is called static search structure organization method [4].

2) Multi Layer Structure

a) Algorithm Description: Denote the root directory as {root}/, and each layer contains k directories representing x bits of the UUID string, so we have: $k=16^x$. To uniquely identify one UUID string, we have to build $m = \left\lceil \frac{32}{x} \right\rceil$ layers of tree structure.

b) Discussion: If we use this structure to organize files, and if the OS accesses all files in a constant time, then the time complexity of inserting, deleting and searching one specific file is O(1).

c) Problems:

- *Layer Number Restriction:* If we create too many layers, will OS support? After testing under windows XP OS, we know that: As long as the total length of the file path isn't longer than MAX_PATH=260 bytes, we can create as many layers as we like to.
- *The Efficiency of File Accessing in Deep Directories:* We will compare the experimental results from structures of different numbers of layer and gave some performance analysis in section 5.

D. Our Organization Method

1) Method Description:

See algorithm description in II-C-2).

For example, if we take x=2, then there're totally m=16 layers, and each layer contains k=256 directories. If we denote each directory as 00, 01, …, ff using 2 bytes, the total file length will certainly not longer than the restriction of MAX_PATH.

See Figure 1: if user "root" uploaded one file "wfms.sql", there was one record inserted into the database as follows:

fileInfoIndex	userIndex	uploadIP	uploadDate	uploadTime	fileName
6e4e9ac0f4934cef99445031825d3322	1	59.67.0.0	2008-09-01	2008-09-01 13:43:29	wfms.sql

Figure 1. Database record.

And if {root} = "D:\uploadRoot", then the file uploaded will be at:

"D:\uploadRoot\9d\e6\f7\65\d1\ee\46\ee\8f\47\b2\95\d9\31\80\3f\bonndeng", see Figure 2:

Figure 2. Uploaded File Location.

As we can see, the name of the file uploaded has become "bonndeng", there're two advantages to do so: first, it's good to unify the file directory length. Secondly, if the file is unauthorized accessed, and the database is safe, the intruder cannot get some original information about the file like original file name and the extension name.

2) Advantages:

- The time complexity is low theoretically.
- There's no need to store the actual location of one uploaded file, because we can locate the file directly by the UUID string.

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on March 05,2024 at 01:34:46 UTC from IEEE Xplore. Restrictions apply.

- We don't have to change the search structure dynamically, which avoids changing the directory structure recursively and improves the efficiency.

III. System Architecture Design and Implementation

A. Service Component Architecture (SCA)

Service Component Architecture (SCA) [5] is a set of specifications which describe a model for building applications and systems using a Service-Oriented Architecture. SCA extends and complements prior approaches to implementing services, and SCA builds on open standards such as Web services.

B. Enterprise Service Bus (ESB)

Enterprise Service Bus (ESB) [6] is an open standards-based distributed synchronous or asynchronous messaging middleware that provides secure interoperability between enterprise applications via XML, Web services interfaces and standardized rules-based routing of documents.

C. Software as a Service (SaaS)

Software as a service (SaaS) [7] is a model of software deployment where an application is hosted as a service provided to customers across the Internet. By eliminating the need to install and run the application on the customer's own computer, SaaS alleviates the customer's burden of software maintenance, ongoing operation, and support.

D. System Architecture Design and Implementation

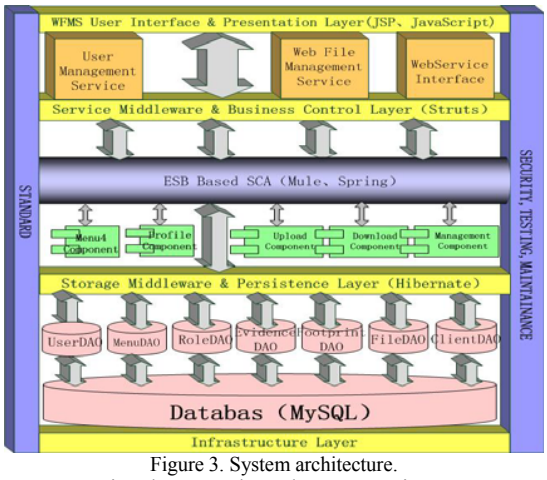

Figure 3. System architecture.

WFMS implemented the Service Component Architecture (SCA) using the technology of the Web page frame (implemented by Struts) and ESB (implemented by Mule which is an open source tool). The architecture of the whole system is shown in Figure 3. From the organizational chart, we can see that this kind of architecture is of distinctive nuance, and it reduces the coupling degree of all the modules, which is more suitable for the code share standard.

As we can see in Figure 3, there're five layers composing the WFMS architecture: The database (MySQL) lays on the Infrastructure Layer, which communicates with the Storage Middleware & Persistence Layer through DAO. And the Storage Middleware & Persistence Layer is managed by the Enterprise Service Bus (ESB). All the components are attached to the ESB and in the charge of Mule and Spring. The ESB is under the control of the Service Middleware & Business Control Layer implemented by Struts, and then forms integrated business logic to serve the upper layers and the Web Service Call. The top layer is the concrete User Interface & Presentation Layer, which is used to present the whole system dynamically to the users and interact with them.

What's worth noting, WFMS uses the MENU component (version 4, developed by the Web and Software Technology R&D center, Tsinghua University, Beijing, China) as the access control module as is shown in Figure 3. The MENU component is packed and released as one independent Web application [8].

Besides, it is remarkable that WFMS supports the SaaS Model. Users can subscribe the service of our system, and use it through the Internet directly without developing and maintaining its own application system, which is of high cost-effective for the consumer. On the other side, SaaS model raises the vitality of our system and is good for our system to expand its field of application.

IV. Functional Module Design and Implementation

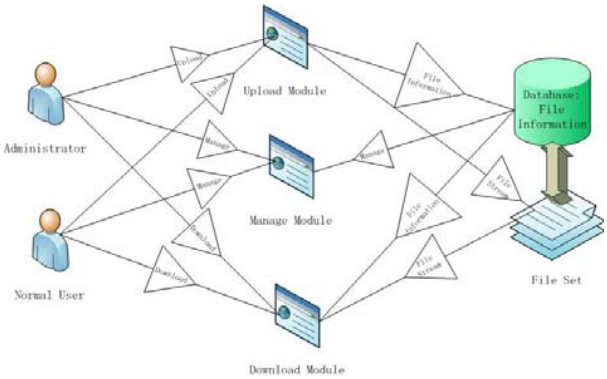

Figure 4. Functional Modules.

As shown in Figure 4, there're three functional modules—administrator and normal user can upload, download files and manage them. But normal users can only manage files uploaded by themselves, while administrators can manage all files uploaded by any user.

V. Experiments

A. Experiment Description

We will compare the experimental results from structures of different numbers of layers. There're two main factors we have to consider: the file size and the number of files. So we will test the relationship between file size and the download speed, and then that between number of files and the download speed.

B. Experiment Result

1) File Size and the Download Speed:

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on March 05,2024 at 01:34:46 UTC from IEEE Xplore. Restrictions apply.

The experiment result is recorded in Table 1 and we can get a corresponding broken line graph from it (Figure 5).

Table 1. File size and the download speed.

DOWN LOAD	26.3KB	263KB	2.57MB	7.72MB	12.8MB	25.7MB
1 layer	0.8019999s	0.87810004s	1.4191s	5.855601s	9.7301s	12.638399s
2 layers	0.733s	0.8703s	1.3546001s	5.8015003s	9.4094s	12.1765s
4 layers	0.8432s	0.84919995s	1.3479999s	6.6066003s	9.151099s	12.427899s
8 layers	0.8132s	0.9092s	1.359s	6.2779s	8.9079s	13.235001s
16 layers	0.76619995s	0.8042s	1.4811001s	6.1088s	8.996s	14.901601s

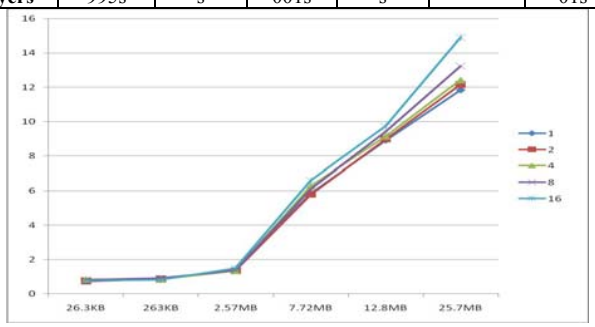

Figure 5. File size and the download speed.

2) Number of Files and the Download Speed:

The experiment result is recorded in Table 2 and we can get a corresponding broken line graph from it (Figure 6).

Table 2. Number of files and the download speed.

DOWN LOAD	10	50	100	200	500	1000
1 layer	0.8019999s	3.3157s	4.0659s	24.323s	29.829666s	101.666s
2 layers	0.733s	3.2294998s	4.1111s	25.8546s	30.779997s	121.338s
4 layers	0.8432s	3.2797997s	4.2571s	26.902s	31.33321s	136.887s
8 layers	0.8132s	3.5772s	4.248s	26.21119s	32.206s	135.339s
16 layers	0.76619995s	3.2668998s	4.3373s	26.3399s	35.345s	140.199s

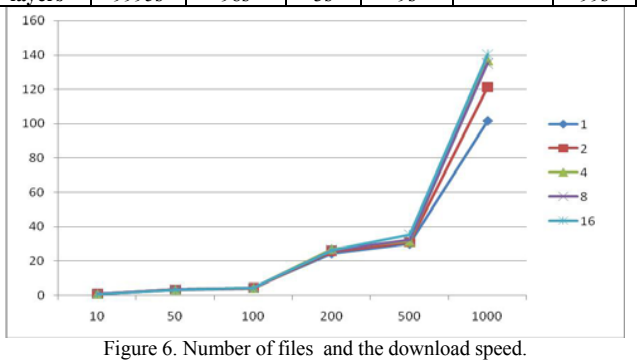

Figure 6. Number of files and the download speed.

VI. Conclusions and the Future Work

A. Conclusions

As we can see in the former section, the bigger the file size is, or the more the files we have to download are, the slower the download speed is (which is so trivial and out of question). But when it comes to the difference caused by the structure of different numbers of layer, we find that, the fewer the number of layer is, the faster the download speed is.

The possible explanation is, Windows locating method uses some data structure like hash table, then the more the number of layer is, the OS need to do more times of hashing, and the slower the download speed is.

On the other side, the number of files under one certain directory is limited. (E.g. As for FAT32 file system, there can be at most 65534 sub directories or files in all under one specific directory.) So we cannot put all files in the same directory, which gets the multi layer structure make sense and gets us to considering the trade-off between the file numbers and the multi layer structure cost.

WFMS can work out an appropriate number of layers to organize files of different orders of magnitude through experiments and analysis.

B. Future Work

The features of WFMS are still not adequate, so at the next stage we'll develop more useful features to supply better online service such as breakpoint continuingly etc.

Secondly, there's a possibility that when more and more websites get to use our system to upload, download and manage Web files, the system itself will become the main bottleneck on the contrary to our original idea of decreasing the burden of websites. When that happens, we'll have to consider the distributed or cluster way of deploying our servers.

Acknowledgment

This research is supported by the Key Technologies R&D Program of China under Grant No. 2006BAH02A12 and the National High-tech R&D Program of China under Grant No. 2006AA010101.

References

[1] China Internet Network Information Center(CNNIC). "The 21st Statistical Report of the Status of China Internet Network Development." 2008.1.

[2] Renkun yin. "Teaching Materials Of Data Structure Course." Department Of Computer Science & Technology In Tsinghua University, Beijing, China. 2006.

[3] Different Part of [2].

[4] Different Part of [2].

[5] Michael Beisiegel, Nickolas Kavantzas, Ashok Malhotra, Greg Pavlik, Chris Sharp. "SCA Service Component Architecture Policy Association Framework Proposal." SCA Collaboration. 2006.

[6] M.-T. Schmidt, B. Hutchison, P. Lambros, R. Phippen. "The enterprise service bus: making service-oriented architecture real." IBM Systems Journal, Oct, 2005.

[7] Dan Ma. "The Business Model of Software-As-A-Service." IEEE International Conference on Services Computing (SCC 2007) pp.701-702. 2007.

[8] Jian Huang, Yong Zhang, Jijiang Yang, Chunxiao Xing. "Design And Implementation Of Web Access Control System Based SCA." Computer Science, volume 34, issue 10B, pp.10-11,20. 2007.

Authorized licensed use limited to: CALIF STATE UNIV NORTHRIDGE. Downloaded on March 05,2024 at 01:34:46 UTC from IEEE Xplore. Restrictions apply.